

Midterm — Sample Exam

Software Engineering — UATX — Spring 2026

Format	Pencil and paper. No computer, phone, notes, or coding agent.
Time	75 minutes (one class).
Length	Five questions, one from each category, weighted equally.
Coverage	Material from weeks 1–4, plus assignments 1, 2, and 3.
Categories	Trace-through, spot the bug, specification, code review, short design.
Grading	Curved. The exam is intentionally hard; expect to leave some parts unfinished.

HOW THIS EXAM THINKS ABOUT AGENTS

The whole semester has leaned into coding agents as the way real engineers work, so an exam that asks you to write working code from scratch on paper would punish you for taking the course seriously. Instead, every question is built around a skill the agent can't do for you: reading code carefully, finding bugs by inspection, writing the spec you'd hand to the agent, judging which of two implementations is better, and making design calls under uncertainty.

HOW TO USE THIS SAMPLE

This is a representative exam, not the actual one — the real exam will have five questions in the same five categories at roughly the same difficulty. Solutions are in a separate document; try the questions cold first, then check.

Question 1

Trace-through

The following module backs a tiny note-taking CLI by storing notes as JSON. Assume `notes.json` does not exist before this script runs.

```
import json
from pathlib import Path

LOG = Path("notes.json")

def load() -> list:
    if not LOG.exists():
        return []
    return json.loads(LOG.read_text())

def save(data: list) -> None:
    LOG.write_text(json.dumps(data))

def add(text: str) -> None:
    data = load()
    data.append({"text": text, "done": False})
    save(data)
```

The following script then runs:

```
add("a")
add("b")
print("A:", len(load()))           # ____

x = load()
x.append({"text": "c", "done": False})
print("B:", len(load()))           # ____

save(x)
print("C:", len(load()))           # ____

y = load()
y[0]["done"] = True
add("d")
print("D:", load()[0]["done"])     # ____

print("E:", [t["text"] for t in load()])  # ____
```

Predict the value printed for each of A, B, C, D, E. For B and D, write one short sentence each explaining *why*.

Question 2

Spot the bug

Your coding agent produced this FastAPI endpoint for listing posts in a forum app. The endpoint passes the obvious tests — `GET /api/posts` returns recent posts, `GET /api/posts?q=hello` returns posts whose body contains "hello", `GET /api/posts?q=hi&limit=10&page=2` returns the second page of ten matches.

```
import os
from fastapi import FastAPI
from sqlalchemy import create_engine, text

app = FastAPI()
engine = create_engine(os.environ.get("DB_URL", "sqlite:///forum.db"))

@app.get("/api/posts")
def list_posts(q: str = "", limit: int = 50, page: int = 1):
    offset = (page - 1) * limit
    if q:
        sql = "SELECT id, author, body FROM posts WHERE body LIKE '%" + q + "%'"
        sql += f" ORDER BY id DESC LIMIT {limit} OFFSET {offset}"
    else:
        sql = (f"SELECT id, author, body FROM posts "
              f"ORDER BY id DESC LIMIT {limit} OFFSET {offset}")
    with engine.connect() as conn:
        rows = conn.execute(text(sql)).all()
    return [{"id": r.id, "author": r.author, "body": r.body} for r in rows]
```

This endpoint has **at least four distinct defects** — some are security holes, some are correctness issues, some are availability issues. Find as many as you can. For *each* defect:

- (a) Give one concrete value of `q`, `limit`, or `page` (or combination) that triggers it.
- (b) In one sentence, describe what goes wrong.
- (c) In one line, describe the fix.

Format as a numbered list, one defect per entry. Two findings that are really the same root cause count once.

Question 3

Specification

You're adding photo albums and shareable links to a photo gallery web app. Users can:

- create a named album, and add or remove photos to it,
- generate a public shareable link to an album (anyone with the link can view, no login required),
- revoke a share link (the URL stops working),
- see a list of their own albums.

Authentication is already handled. Assume the caller's identity is available via an `X-Username` header.

Spec the REST API. Cover:

- Every endpoint you'd add: HTTP method, URL, request body, response body, status codes for success and the obvious failure modes.
- For at least one of your endpoints, whether it is idempotent, and why that matters.
- One design call that's not obvious, with one sentence on why.

Question 4

Code review

Two implementations of the same function. Both are intended to return a new list of items flagged as expired (timestamp older than the cutoff), with an `expired: True` key added to each — leaving the original input list untouched.

Version A:

```
def flag_expired(items, cutoff):
    result = []
    for i in range(len(items)):
        if items[i]["timestamp"] < cutoff:
            items[i]["expired"] = True
            result.append(items[i])
    return result
```

Version B:

```
def flag_expired(items, cutoff):
    return [
        {**item, "expired": True}
        for item in items
        if item["timestamp"] < cutoff
    ]
```

Pick one. Justify in three to five sentences, and call out anything that's actually broken in either version.

Question 5

Short design

Your e-commerce app's mobile client lives on flaky cell networks. When a user taps "place order" and the request times out, the client retries automatically. You're designing two endpoints: `POST /api/orders` (place an order, charges payment, decrements inventory) and `PATCH /api/orders/{id}/shipping` (update the shipping address before the order ships).

Which of these endpoints is idempotent under retry, and which one isn't? For the one that isn't, sketch one specific design move you could make so a retry doesn't double-charge or double-decrement. One paragraph total.